Kth Smallest

Difficulty: **Medium** Accuracy: **35.17%** Submissions: **736K+** Points: **4**

Average Time: **25m**

Given an integer array **arr[]** and an integer **k**, your task is to find and return the **k**th **smallest** element in the given array.

Note: The kth smallest element is determined based on the sorted order of the array.

Examples :

Java (21) Start Timer

```java
import java.util.*;

class Solution {
    public static int kthSmallest(int[] arr, int k) {
        PriorityQueue<Integer> pq = new PriorityQueue<>();
        for (int x : arr) {
            pq.add(x);
        }
        for (int i = 1; i < k; i++) {
            pq.poll();
        }
        return pq.peek();
    }

    public static void main(String[] args) {
        int[] arr1 = {10, 5, 4, 3, 48, 6, 2, 33, 53, 10};
        int k1 = 4;
        System.out.println(kthSmallest(arr1, k1));

        int[] arr2 = {7, 10, 4, 3, 20, 15};
        int k2 = 3;
        System.out.println(kthSmallest(arr2, k2));
    }
}
```

Custom Input Compile & Run Submit

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1121 / 1121	**2 / 2**

Minimize the Heights II

Difficulty: **Medium** Accuracy: **15.06%** Submissions: **770K+** Points: **4**

Average Time: **25m**

Given an array **arr[]** denoting heights of **n** towers and a positive integer **k**.

For **each** tower, you must perform **exactly one** of the following operations **exactly once**.

- **Increase** the height of the tower by **k**
- **Decrease** the height of the tower by **k**

Java (21) Start Timer

```java
import java.util.*;

class Solution {
    public static int getMinDiff(int[] arr, int k) {
        int n = arr.length;
        Arrays.sort(arr);

        int ans = arr[n - 1] - arr[0];

        for (int i = 1; i < n; i++) {
            if (arr[i] - k < 0) continue;

            int minVal = Math.min(arr[0] + k, arr[i] - k);
            int maxVal = Math.max(arr[n - 1] - k, arr[i - 1] + k);

            ans = Math.min(ans, maxVal - minVal);
        }

        return ans;
    }

    public static void main(String[] args) {
        int[] arr1 = {1, 5, 8, 10};
        int k1 = 2;
        System.out.println(getMinDiff(arr1, k1));

        int[] arr2 = {3, 9, 12, 16, 20};
        int k2 = 3;
        System.out.println(getMinDiff(arr2, k2));
    }
}
```

Custom Input Compile & Run Submit

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1115 / 1115	**1 / 2**

Minimum Jumps

Difficulty: **Medium** Accuracy: **11.91%** Submissions: **1.1M** Points: **4**

You are given an array **arr[]** of non-negative numbers. Each number tells you the **maximum number of steps** you can jump forward from that position.

For example:

- If **arr[i] = 3**, you can jump to index **i + 1**, **i + 2**, or **i + 3** from position **i**.
- If **arr[i] = 0**, you **cannot jump forward** from that position.

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1120 / 1120	**2 / 2**

Java (21) Start Timer

```java
class Solution {
    public static int minJumps(int[] arr) {
        int n = arr.length;
        if (n <= 1) return 0;
        if (arr[0] == 0) return -1;

        int jumps = 1;
        int maxReach = arr[0];
        int steps = arr[0];

        for (int i = 1; i < n; i++) {
            if (i == n - 1) return jumps;

            maxReach = Math.max(maxReach, i + arr[i]);
            steps--;

            if (steps == 0) {
                jumps++;
                if (i >= maxReach) return -1;
                steps = maxReach - i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] a1 = {1, 3, 5, 8, 9, 2, 6, 7, 6, 8, 9};
        System.out.println(minJumps(a1));

        int[] a2 = {1, 4, 3, 2, 6, 7};
        System.out.println(minJumps(a2));
```

Custom Input Compile & Run Submit

leetcode.com/problems/merge-intervals/description/

Problem List | Submit | Premium

Description | Editorial | Solutions | Submissions

56. Merge Intervals

Solved

Medium | Topics | Companies

Given an array of `intervals` where `intervals[i] = [start_i, end_i]`, merge all overlapping intervals, and return *an array of the non-overlapping intervals that cover all the intervals in the input.*

Example 1:

```
Input: intervals = [[1,3],[2,6],[8,10],[15,18]]
Output: [[1,6],[8,10],[15,18]]
Explanation: Since intervals [1,3] and [2,6] overlap, merge them into [1,6].
```

Example 2:

```
Input: intervals = [[1,4],[4,5]]
Output: [[1,5]]
```

24.3K | 263 | 396 Online

Code

Java | Auto

```java
import java.util.*;

class Solution {
    public int[][] merge(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
        List<int[]> res = new ArrayList<>();

```

Saved | Ln 1, Col 1

Testcase | Test Result

Accepted Runtime: 1 ms

Case 1 | Case 2 | Case 3

Input

intervals =

[[1,3],[2,6],[8,10],[15,18]]

Merge Without Extra Space

Difficulty: **Medium** Accuracy: **32.01%** Submissions: **326K+** Points: **4**

Average Time: **20m**

Given two sorted arrays **a[]** and **b[]** of size **n** and **m** respectively, the task is to merge them in sorted order without using any **extra space**. Modify **a[]** so that it contains the first **n** elements and modify **b[]** so that it contains the last **m** elements.

Examples:

Input: a[] = [2, 4, 7, 10], b[] = [2, 3]

Output Window

Compilation Results | Custom Input | Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1111 / 1111	**1 / 1**

Java (21) | Start Timer

```java
            while (j < n + m) {
                if (i < n && j < n) {
                    if (a[i] > a[j]) {
                        int temp = a[i];
                        a[i] = a[j];
                        a[j] = temp;
                    }
                } else if (i < n && j >= n) {
                    if (a[i] > b[j - n]) {
                        int temp = a[i];
                        a[i] = b[j - n];
                        b[j - n] = temp;
                    }
                } else {
                    if (b[i - n] > b[j - n]) {
                        int temp = b[i - n];
                        b[i - n] = b[j - n];
                        b[j - n] = temp;
                    }
                }
                i++;
                j++;
            }

            if (gap == 1) break;
            gap = (gap + 1) / 2;
        }
    }
}
```

Custom Input | Compile & Run | Submit

leetcode.com/problems/merge-intervals/description/

Problem List | Submit | Premium

Description | Accepted | Editorial | Solutions | Submissions

56. Merge Intervals

Solved

Medium | Topics | Companies

Given an array of `intervals` where `intervals[i] = [start_i, end_i]`, merge all overlapping intervals, and return *an array of the non-overlapping intervals that cover all the intervals in the input.*

Example 1:

```
Input: intervals = [[1,3],[2,6],[8,10],[15,18]]
Output: [[1,6],[8,10],[15,18]]
Explanation: Since intervals [1,3] and [2,6] overlap,
merge them into [1,6].
```

Example 2:

```
Input: intervals = [[1,4],[4,5]]
Output: [[1,5]]
```

24.3K | 263 | 397 Online

Code

Java | Auto

```java
import java.util.*;

class Solution {
    public int[][] merge(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
        List<int[]> res = new ArrayList<>();

```

Saved | Ln 25, Col 1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

intervals =

[[1,3],[2,6],[8,10],[15,18]]

Common in 3 Sorted Arrays

Difficulty: **Easy** Accuracy: **22.16%** Submissions: **439K+** Points: **2**

Given three sorted arrays in **non-decreasing** order, print all common elements in **non-decreasing** order across these arrays. If there are no such elements return an empty array. In this case, the output will be -1.

Note: can you handle the duplicates without using any additional Data Structure?

Examples :

Java (21) Start Timer

```java
class Solution {
    public static ArrayList<Integer> commonElements(int[] arr1, int[] arr2,
        ArrayList<Integer> res = new ArrayList<>();
        int i = 0, j = 0, k = 0;

        while (i < arr1.length && j < arr2.length && k < arr3.length) {
            if (arr1[i] == arr2[j] && arr2[j] == arr3[k]) {
                if (res.size() == 0 || res.get(res.size() - 1) != arr1
                    res.add(arr1[i]);
                }
                i++;
                j++;
                k++;
            } else if (arr1[i] < arr2[j]) {
                i++;
            } else if (arr2[j] < arr3[k]) {
                j++;
            } else {
                k++;
            }
        }

        if (res.size() == 0) {
            res.add(-1);
        }

        return res;
    }
}
```

Custom Input Compile & Run Submit

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Compilation Error

Ask Yogi Bot

```
./GFG.java: 31: error: incompatible types: List<Integer> cannot be converted to int[]
        List<Integer> res = ob.commonElements(arr, brr, crr);
                                                        ^
```

Factorials of large numbers

Difficulty: **Medium** Accuracy: **36.57%** Submissions: **177K+** Points: **4**

Average Time: **20m**

Given an integer **n,** find its factorial. Return a list of integers denoting the digits that make up the factorial of n.

Examples:

Input: n = 5

Output: [1, 2, 0]

Java (21) Start Timer

```java
import java.util.*;

class Solution {
    static ArrayList<Integer> factorial(int n) {
        ArrayList<Integer> res = new ArrayList<>();
        res.add(1);

        for (int x = 2; x <= n; x++) {
            int carry = 0;
            for (int i = 0; i < res.size(); i++) {
                int val = res.get(i) * x + carry;
                res.set(i, val % 10);
                carry = val / 10;
            }
            while (carry > 0) {
                res.add(carry % 10);
                carry /= 10;
            }
        }

        Collections.reverse(res);
        return res;
    }
}
```

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1111 / 1111	**1 / 1**

Custom Input Compile & Run Submit

geeksforgeeks.org/problems/array-subset-of-anotherarray2317/1

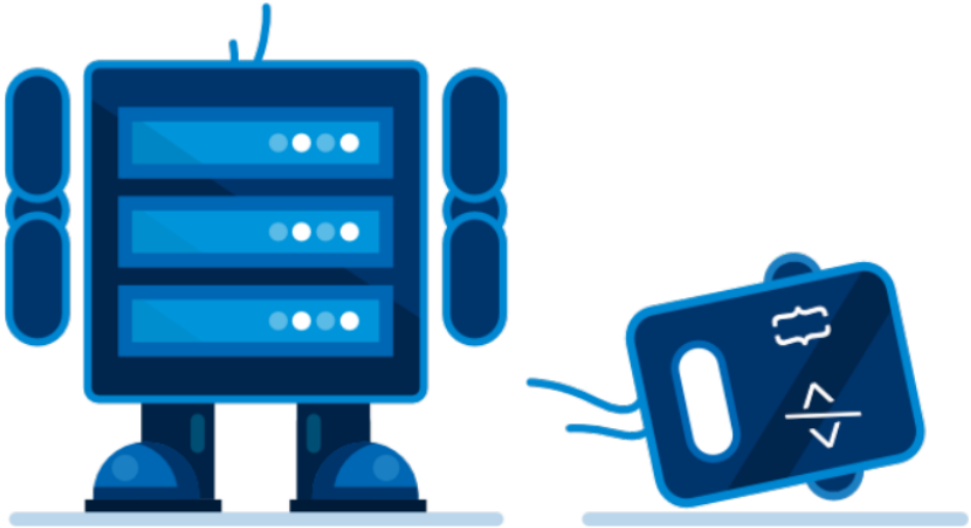
Search...

Three 90 Ending

Courses Tutorials Practice Jobs

Oops!! Something went wrong.

Not to worry we will be up and running shortly.
You may try again later or report this issue at
support@geeksforgeeks.org



Go back to Homepage

Triplet Sum in Array

Difficulty: **Medium** Accuracy: **35.0%** Submissions: **360K+** Points: **4** Average Time: **15m**

Given an array **arr[]** and an integer **target**, determine if there exists a triplet in the array whose sum equals the given **target**.

Return **true** if such a triplet exists, otherwise, return **false**.

Examples:

Input: arr[] = [1, 4, 45, 6, 10, 8], target = 13

Java (21) | Start Timer

```java
import java.util.*;

class Solution {
    public boolean hasTripletSum(int[] arr, int target) {
        Arrays.sort(arr);
        int n = arr.length;

        for (int i = 0; i < n - 2; i++) {
            int l = i + 1, r = n - 1;
            int need = target - arr[i];

            while (l < r) {
                int sum = arr[l] + arr[r];
                if (sum == need) return true;
                if (sum < need) l++;
                else r--;
            }
        }
        return false;
    }
}
```

Custom Input | Compile & Run | Submit

Output Window

Compilation Results | Custom Input | Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1111 / 1111	**1 / 1**

Trapping Rain Water

Difficulty: **Hard** Accuracy: **33.14%** Submissions: **497K+** Points: **8** Average Time: **20m**

Given an array **arr[]** with non-negative integers representing the height of blocks. If the width of each block is 1, compute how much water can be trapped between the blocks during the rainy season.

Examples:

Input: arr[] = [3, 0, 1, 0, 4, 0 2]
Output: 10

Java (21) Start Timer

```java
class Solution {
    public int maxWater(int[] arr) {
        int n = arr.length;
        int left = 0, right = n - 1;
        int leftMax = 0, rightMax = 0;
        int water = 0;

        while (left < right) {
            if (arr[left] <= arr[right]) {
                if (arr[left] >= leftMax) {
                    leftMax = arr[left];
                } else {
                    water += leftMax - arr[left];
                }
                left++;
            } else {
                if (arr[right] >= rightMax) {
                    rightMax = arr[right];
                } else {
                    water += rightMax - arr[right];
                }
                right--;
            }
        }
        return water;
    }
}
```

Custom Input Compile & Run Submit

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1111 / 1111	**1 / 1**